

第 17 章

图像分割与提取

在图像处理的过程中，经常需要从图像中将前景对象作为目标图像分割或者提取出来。例如，在视频监控中，观测到的是固定背景下的视频内容，而我们对背景本身并无兴趣，感兴趣的是背景中出现的车辆、行人或者其他对象。我们希望通过将这些对象从视频中提取出来，而忽略那些没有对象进入背景的视频内容。

在前面的章节中，我们讨论了如何使用诸如图像形态学变换、阈值算法、图像金字塔、图像轮廓、边缘检测等方法对图像进行分割。本章介绍使用分水岭算法及 GrabCut 算法对图像进行分割及提取。

17.1 用分水岭算法实现图像分割与提取

图像分割是图像处理过程中一种非常重要的操作。分水岭算法将图像形象地比喻为地理学上的地形表面，实现图像分割，该算法非常有效。

17.1.1 算法原理

冈萨雷斯在《数字图像处理》一书中，对分水岭算法进行了细致的分析与介绍。OpenCV 的官网建议学习者阅读国立巴黎高等矿业学校图像处理实验室（The Image Processing Laboratory of MINES ParisTech）的 CMM（Centre for Mathematical Morphology）网站上关于分水岭算法的介绍和动画演示。

下面对分水岭算法的相关内容做简单的介绍。

任何一幅灰度图像，都可以被看作是地理学上的地形表面，灰度值高的区域可以被看成是山峰，灰度值低的区域可以被看成是山谷。如图 17-1 所示，其中左图是原始图像，右图是其对应的“地形表面”。

如果我们向每一个山谷中“灌注”不同颜色的水（这里采用了 OpenCV 官网的表述，冈萨雷斯将灌注表述为在山谷中打洞，然后让水穿过洞以均匀的速率上升）。那么，随着水位不断地升高，不同山谷的水就会汇集到一起。在这个过程中，为了防止不同山谷的水交汇，我们需要在水流可能汇合的地方构建堤坝。该过程将图像分成两个不同的集合：集水盆地和分水岭线。我们构建的堤坝就是分水岭线，也即对原始图像的分割。这就是分水岭算法。



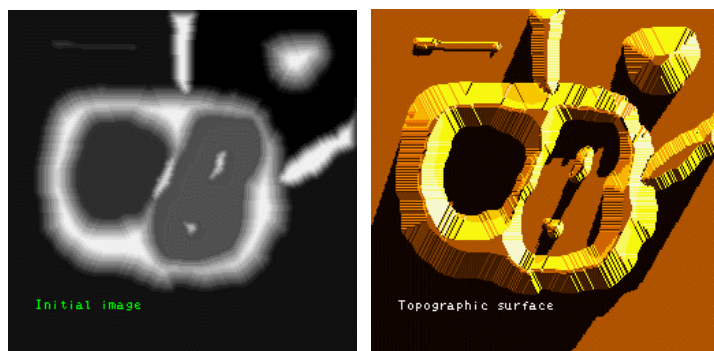


图 17-1 灰度图及其对应的“地形表面”

在图 17-2 中，左图是原始图像，右图是使用分水岭算法得到的图像分割结果。在 CMM 的网站上不仅提供了该示例图像，还提供了动画演示效果，有兴趣的读者可以去网站上看看。

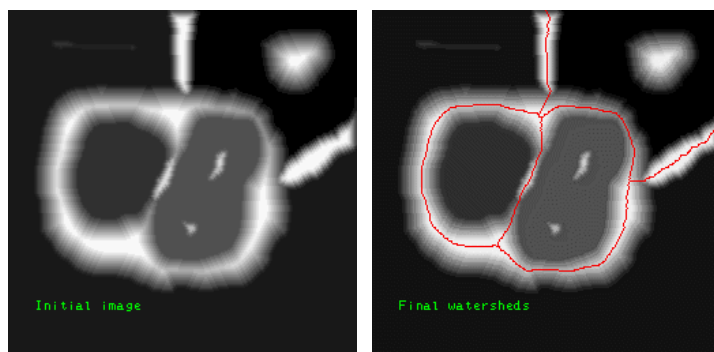


图 17-2 图像分割的效果演示

由于噪声等因素的影响，采用上述基础分水岭算法经常会得到过度分割的结果。过度分割会将图像划分为一个个稠密的独立小块，让分割失去了意义。图 17-3 展示了过度分割的图像。其中左图是电泳现象的图像，右图是过度分割的结果图像，可以看到过度分割现象非常严重。

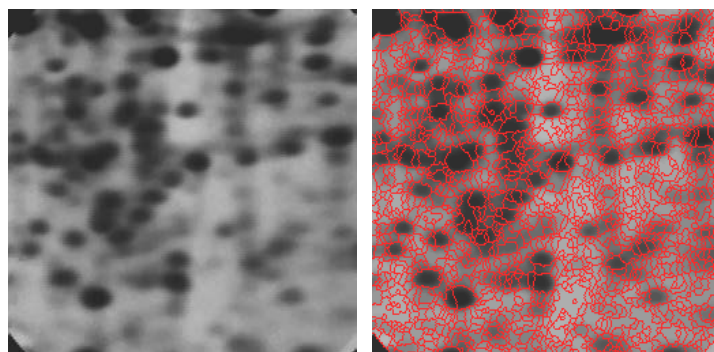


图 17-3 对电泳图像的过度分割结果

为了改善图像分割效果，人们提出了基于掩模的改进的分水岭算法。改进的分水岭算法允许用户将他认为是同一个分割区域的部分标注出来（被标注的部分就称为掩模）。这样，分水岭算法在处理时，就会将标注的部分处理为同一个分割区域。大家可以尝试使用微软 PowerPoint 中的“删除背景”功能，加深对此改进算法的理解。

在图 17-4 中，左图是原始图像，我们对其做了标注处理，其中被标注为深色的三个小块表示：在使用掩模分水岭算法时，这些部分所包含的颜色都会被分割在同一个区域内。使用掩模分水岭算法得到的分割结果如图 17-4 中的右图所示。

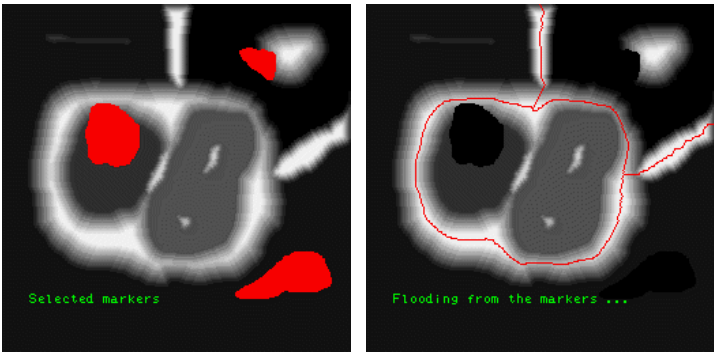


图 17-4 使用了掩模的分水岭算法

采用改进的分水岭算法对图 17-5 中左侧的电泳图像进行掩模处理,得到右侧的分割结果。可以看出，分割结果得到明显的改进。

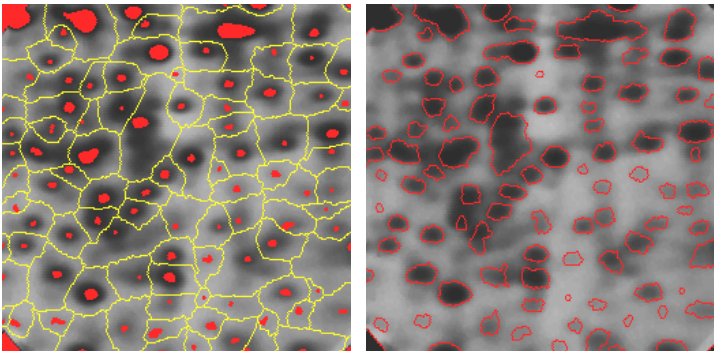


图 17-5 采用改进的分水岭算法重新分割电泳图像

17.1.2 相关函数介绍

在 OpenCV 中，可以使用函数 `cv2.watershed()`实现分水岭算法。在具体的实现过程中，还需要借助于形态学函数、距离变换函数 `cv2.distanceTransform()`、`cv2.connectedComponents()`来完成图像分割。下面对分水岭算法中用到的函数进行简单的说明。

1. 形态学函数回顾

在使用分水岭算法对图像进行分割前，需要对图像进行简单的形态学处理。先回顾一下形态学里的基本操作。

(1) 开运算

开运算是先腐蚀、后膨胀的操作，开运算能够去除图像内的噪声。例如，在图 17-6 中，先对左图进行腐蚀操作，会得到中间的图像，再对中间的图像进行膨胀操作，会得到右侧的图像。对左图进行开运算（先腐蚀、后膨胀）后，我们得到了右图。通过观察可知，左图在经过开运算后变成右图以后，上面的毛刺（噪声信息）已经被去除了。



图 17-6 开运算示意图

对图像进行开运算，能够去除图像内的噪声。在用分水岭算法处理图像前，要先使用开运算去除图像内的噪声，以避免噪声对图像分割可能造成的干扰。

(2) 获取图像边界

通过形态学操作和减法运算能够获取图像的边界。例如，在图 17-7 中，左图是原始图像，中间的图是对其进行腐蚀而得到的图像，对二者进行减法运算，就会得到右侧的图像。通过观察可知，右图是左图的边界。



图 17-7 获取图像边界

【例 17.1】使用形态学变换，获取一幅图像的边界信息，并观察效果。

根据题目的要求，编写代码如下：

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
o=cv2.imread("my.bmp",cv2.IMREAD_UNCHANGED)
k=np.ones((5,5),np.uint8)
e=cv2.erode(o,k)
b=cv2.subtract(o,e)
plt.subplot(131)
plt.imshow(o)
plt.axis('off')
plt.subplot(132)
plt.imshow(e)
plt.axis('off')
plt.subplot(133)
plt.imshow(b)
plt.axis('off')
```

运行上述程序，得到结果如图 17-8 所示，其中左图是原始图像，中间的图是对其进行腐蚀而得到的图像，右图是原始图像减去腐蚀图像后得到的边界图像。可以看到，右图比较准确地显示出了左图内前景对象的边界信息。

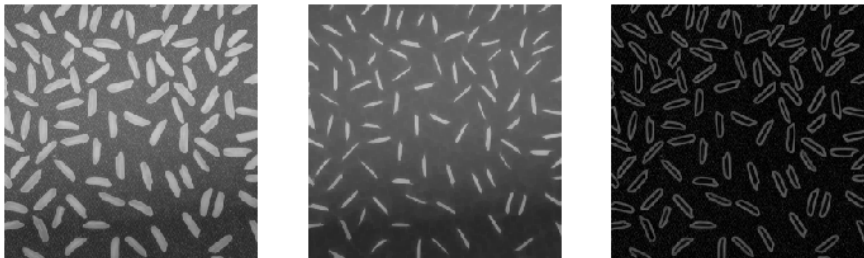


图 17-8 利用形态学变换获取图像边界

通过以上分析可知，使用形态学操作和减法运算能够获取图像的边界信息。但是，形态学操作仅适用于比较简单的图像。如果图像内的前景对象存在连接的情况，使用形态学操作就无法准确获取各个子图像的边界了。

2. 距离变换函数distanceTransform

当图像内的各个子图没有连接时，可以直接使用形态学的腐蚀操作确定前景对象，但是如果图像内的子图连接在一起时，就很难确定前景对象了。此时，借助于距离变换函数 cv2.distanceTransform()可以方便地将前景对象提取出来。

距离变换函数 cv2.distanceTransform()计算二值图像内任意点到最近背景点的距离。一般情况下，该函数计算的是图像内非零值像素点到最近的零值像素点的距离，即计算二值图像中所有像素点距离其最近的值为 0 的像素点的距离。当然，如果像素点本身的值为 0，则这个距离也为 0。

距离变换函数 cv2.distanceTransform()的计算结果反映了各个像素与背景(值为 0 的像素点)的距离关系。通常情况下：

- 如果前景对象的中心（质心）距离值为 0 的像素点距离较远，会得到一个较大的值。
- 如果前景对象的边缘距离值为 0 的像素点较近，会得到一个较小的值。

如果对上述计算结果进行阈值化，就可以得到图像内子图的中心、骨架等信息。距离变换函数 cv2.distanceTransform()可以用于计算对象的中心，还能细化轮廓、获取图像前景等，有多种功能。

距离变换函数 cv2.distanceTransform()的语法格式为：

```
dst=cv2.distanceTransform(src, distanceType, maskSize[, dstType])
```

式中：

- src 是 8 位单通道的二值图像。
- distanceType 为距离类型参数，其具体值和含义如表 17-1 所示。

表 17-1 distanceType 参数的值及含义

参数值	含义
cv2.DIST_USER	用户自定义距离。
cv2.DIST_L1	$distance = x1-x2 + y1-y2 $
cv2.DIST_L2	简单欧几里得距离（欧氏距离）
cv2.DIST_C	$distance = \max(x1-x2 , y1-y2)$

续表

参数值	含义
cv2.DIST_L12	L1-L2 metric: $\text{distance} = 2(\sqrt{1+x^2/2} - 1)$
cv2.DIST_FAIR	$\text{distance} = c^2(x /c - \log(1+ x /c))$, $c = 1.3998$
cv2.DIST_WELSCH	$\text{distance} = c^2/2(1 - \exp(-(x/c)^2))$, $c = 2.9846$
cv2.DIST_HUBER	$\text{distance} = x < c ? x^2/2 : c(x - c/2)$, $c = 1.345$

- maskSize 为掩模的尺寸，其可能的值如表 17-2 所示。需要注意，当 distanceType = cv2.DIST_L1 或 cv2.DIST_C 时，maskSize 强制为 3（因为设置为 3 和设置为 5 及更大值没有什么区别）。

表 17-2 maskSize 的值

参数值	对应整数值
cv2.DIST_MASK_3	3
cv2.DIST_MASK_5	5
cv2.DIST_MASK_PRECISE	

- dstType 为目标图像的类型，默认值为 CV_32F。
- dst 表示计算得到的目标图像，可以是 8 位或 32 位浮点数，尺寸和 src 相同。

【例 17.2】使用距离变换函数 cv2.distanceTransform()，计算一幅图像的确定前景，并观察效果。

根据题目的要求，编写代码如下：

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
img = cv2.imread('water_coins.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
img=cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
ishow=img.copy()
ret, thresh = cv2.threshold(gray,0,255,cv2.THRESH_BINARY_INV+cv2.THRESH_OTSU)
kernel = np.ones((3,3),np.uint8)
opening = cv2.morphologyEx(thresh,cv2.MORPH_OPEN,kernel, iterations = 2)
dist_transform = cv2.distanceTransform(opening,cv2.DIST_L2,5)
ret, fore = cv2.threshold(dist_transform,0.7*dist_transform.max(),255,0)
plt.subplot(131)
plt.imshow(ishow)
plt.axis('off')
plt.subplot(132)
plt.imshow(dist_transform)
plt.axis('off')
plt.subplot(133)
plt.imshow(fore)
plt.axis('off')
```

运行上述程序，得到结果如图 17-9 所示。其中：



- 左图是原始图像。
- 中间的是距离变换函数 `cv2.distanceTransform()` 计算得到的距离图像。
- 右图是对距离图像进行阈值化处理后的结果图像。

从图 17-9 可以看到，右图比较准确地显示出左图内的“确定前景”。这里的确定前景，通常是指前景对象的中心。之所以认为这些点是确定前景，是因为它们距离背景点的距离足够远，都是距离大于足够大的固定阈值（`0.7*dist_transform.max()`）的点。

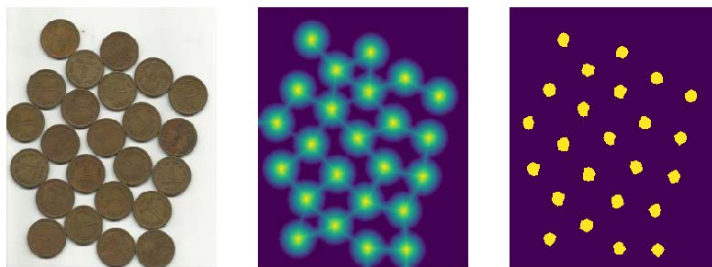


图 17-9 【例 17.2】程序的运行结果

3. 确定未知区域

使用形态学的膨胀操作能够将图像内的前景“膨胀放大”。当图像内的前景被放大后，背景就会被“压缩”，所以此时得到的背景信息一定小于实际背景的，不包含前景的“确定背景”。以下为了方便说明将确定背景称为 B。

距离变换函数 `cv2.distanceTransform()` 能够获取图像的“中心”，得到“确定前景”。为了方便说明，将确定前景称为 F。

图像中有了确定前景 F 和确定背景 B，剩下区域的就是未知区域 UN 了。这部分区域正是分水岭算法要进一步明确的区域。

针对一幅图像 O，通过以下关系能够得到未知区域 UN：

$$\text{未知区域 UN} = \text{图像 O} - \text{确定背景 B} - \text{确定前景 F}$$

对上述表达式进行整理，可以得到：

$$\text{未知区域 UN} = (\text{图像 O} - \text{确定背景 B}) - \text{确定前景 F}$$

上式中的“图像 O - 确定背景 B”，可以通过对图像进行形态学的膨胀操作得到。

【例 17.3】 标注一幅图像的确定前景、确定背景及未知区域。

根据题目的要求，编写代码如下：

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
img = cv2.imread('water_coins.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
imshow = img.copy()
ret, thresh = cv2.threshold(gray, 0, 255,
```

```

cv2.THRESH_BINARY_INV+cv2.THRESH_OTSU)
kernel = np.ones((3,3),np.uint8)
opening = cv2.morphologyEx(thresh,cv2.MORPH_OPEN,kernel, iterations = 2)
bg = cv2.dilate(opening,kernel,iterations=3)
dist = cv2.distanceTransform(opening,cv2.DIST_L2,5)
ret, fore = cv2.threshold(dist,0.7*dist.max(),255,0)
fore = np.uint8(fore)
un = cv2.subtract(bg,fore)
plt.subplot(221)
plt.imshow(ishow)
plt.axis('off')
plt.subplot(222)
plt.imshow(bg)
plt.axis('off')
plt.subplot(223)
plt.imshow(fore)
plt.axis('off')
plt.subplot(224)
plt.imshow(un)
plt.axis('off')

```

运行上述程序，得到的结果如图 17-10 所示。其中：

- 左上角是原始图像 ishow。
- 右上角是对图像 ishow 进行膨胀后得到的图像 bg，其背景图像是确定背景，前景图像是“原始图像-确定背景”。
- 左下角是确定前景图像 fore。
- 右下角图像中的小圆环就是未知区域图像 un，是由图像 bg 和图像 fore 相减得到的。也就是说，未知区域图像 un 来源于“原始图像-确定背景-确定前景”。

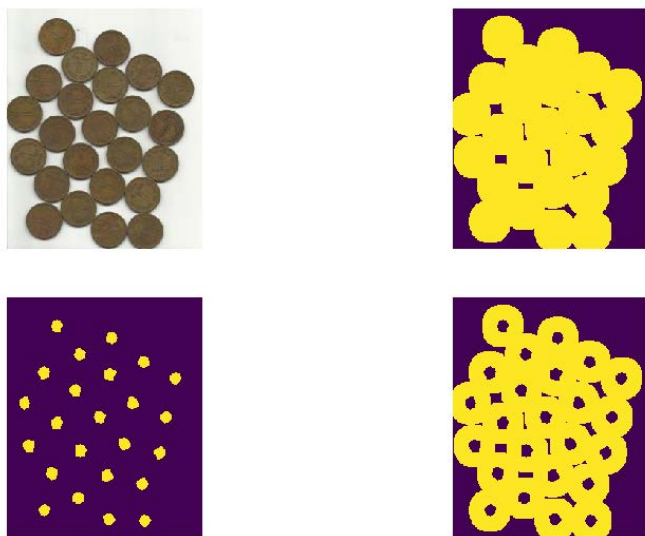


图 17-10 【例 17.3】程序的运行结果

值得注意的是，在图 17-10 右上角的图像 bg 中：

- 前景的一个个小圆是“原始图像 - 确定背景”部分，而不是“确定背景”。
- 其背景图像才是“确定背景”。

4. 函数 connectedComponents

明确了确定前景后，就可以对确定前景图像进行标注了。在 OpenCV 中，可以使用函数 `cv2.connectedComponents()` 进行标注。该函数会将背景标注为 0，将其他的对象使用从 1 开始的正整数标注。

函数 `cv2.connectedComponents()` 的语法格式为：

```
retval, labels = cv2.connectedComponents( image )
```

式中：

- `image` 为 8 位单通道的待标注图像。
- `retval` 为返回的标注的数量。
- `labels` 为标注的结果图像。

【例 17.4】 使用函数 `cv2.connectedComponents()` 标注一幅图像，并观察标注的效果。

根据题目的要求，编写代码如下：

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
img = cv2.imread('water_coins.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
ishow = img.copy()
ret, thresh = cv2.threshold(gray, 0, 255,
                             cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
kernel = np.ones((3, 3), np.uint8)
opening = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel, iterations = 2)
sure_bg = cv2.dilate(opening, kernel, iterations=3)
dist_transform = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
ret, fore = cv2.threshold(dist_transform, 0.7 * dist_transform.max(), 255, 0)
fore = np.uint8(fore)
ret, markers = cv2.connectedComponents(fore)
plt.subplot(131)
plt.imshow(ishow)
plt.axis('off')
plt.subplot(132)
plt.imshow(fore)
plt.axis('off')
plt.subplot(133)
plt.imshow(markers)
plt.axis('off')
print(ret)
```

运行上述程序，得到的结果如图 17-11 所示。其中：

- 左图是原始图像 `ishow`。
- 中间的是经过距离变换后得到的前景图像的中心点图像 `fore`。
- 右图是对前景图像的中心点图像进行标注后的结果图像 `markers`。

可以看到，前景图像的中心点被做了不同的标注（用不同颜色区分，在纸质书中显示为不同的灰度）。

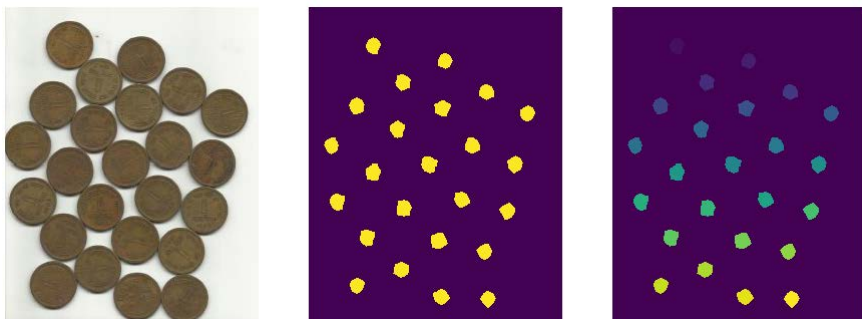


图 17-11 标注结果

函数 `cv2.connectedComponents()` 在标注图像时，会将背景标注为 0，将其他的对象用从 1 开始的正整数标注。具体的对应关系为：

- 数值 0 代表背景区域。
- 从数值 1 开始的值，代表不同的前景区域。

在分水岭算法中，标注值 0 代表未知区域。所以，我们要对函数 `cv2.connectedComponents()` 标注的结果进行调整：将标注的结果都加上数值 1。经过上述处理后，在标注结果中：

- 数值 1 代表背景区域。
- 从数值 2 开始的值，代表不同的前景区域。

为了能够使用分水岭算法，还需要对原始图像内的未知区域进行标注，将已经计算出来的未知区域标注为 0 即可。

这里的关键代码为：

```
ret, markers = cv2.connectedComponents(fore)
markers = markers+1
markers[未知区域] = 0
```

【例 17.5】使用函数 `cv2.connectedComponents()` 标注一幅图像，并对其进行修正，使未知区域被标注为 0，并观察标注的效果。

根据题目的要求，编写代码如下：

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
img = cv2.imread('water_coins.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```

img=cv2.cvtColor(img,cv2.COLOR_BGR2RGB)
imshow=img.copy()
ret, thresh = cv2.threshold(gray,0,255,
                           cv2.THRESH_BINARY_INV+cv2.THRESH_OTSU)
kernel = np.ones((3,3),np.uint8)
opening = cv2.morphologyEx(thresh,cv2.MORPH_OPEN,kernel, iterations = 2)
sure_bg = cv2.dilate(opening,kernel,iterations=3)
dist_transform = cv2.distanceTransform(opening,cv2.DIST_L2,5)
ret, fore = cv2.threshold(dist_transform,0.7*dist_transform.max(),255,0)
fore = np.uint8(fore)
ret, markers1 = cv2.connectedComponents(fore)
foreAdv=fore.copy()
unknown = cv2.subtract(sure_bg,foreAdv)
ret, markers2 = cv2.connectedComponents(foreAdv)
markers2 = markers2+1
markers2[unknown==255] = 0
plt.subplot(121)
plt.imshow(markers1)
plt.axis('off')
plt.subplot(122)
plt.imshow(markers2)
plt.axis('off')

```

运行上述程序，得到的结果如图 17-12 所示。其中：

- 左图是对一幅图像使用函数 `cv2.connectedComponents()` 直接标注后的结果。
- 右图是修正后的标注结果。

对比左右图可以看出，右图在前景图像的边缘（未知区域）进行了标注，使得每一个确定前景都有一个黑色的边缘，这个边缘是被标注的未知区域。由于图像对比度的原因，在纸质书上观察到的效果可能并不明显，大家可以运行这段代码，在计算机屏幕上观察结果。

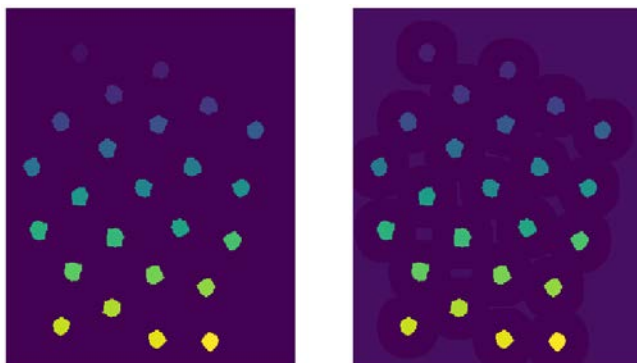


图 17-12 标注效果的对比

4. 函数 `cv2.watershed()`

完成上述处理后，就可以使用分水岭算法对预处理结果图像进行分割了。在 OpenCV 中，实现分水岭算法的函数是 `cv2.watershed()`，其语法格式为：



```
markers = cv2.watershed( image, markers )
```

式中:

- `image` 是输入图像, 必须是 8 位三通道的图像。在对图像使用 `cv2.watershed()` 函数处理之前, 必须先用正数大致勾画出图像中的期望分割区域。每一个分割的区域会被标注为 1、2、3 等。对于尚未确定的区域, 需要将它们标注为 0。我们可以将标注区域理解为进行分水岭算法分割的“种子”区域。
- `markers` 是 32 位单通道的标注结果, 它应该和 `image` 具有相等大小。在 `markers` 中, 每一个像素要么被设置为初期的“种子值”, 要么被设置为“-1”表示边界。`markers` 可以省略。

17.1.3 分水岭算法图像分割实例

本节结合前面介绍的知识, 讲解一个图像分割实例。使用分水岭算法进行图像分割时, 基本的步骤为:

1. 通过形态学开运算对原始图像 `O` 去噪。
2. 通过腐蚀操作获取“确定背景 `B`”。需要注意, 这里得到“原始图像-确定背景”即可。
3. 利用距离变换函数 `cv2.distanceTransform()` 对原始图像进行运算, 并对其进行阈值处理, 得到“确定前景 `F`”。
4. 计算未知区域 `UN` ($UN = O - B - F$)。
5. 利用函数 `cv2.connectedComponents()` 对原始图像 `O` 进行标注。
6. 对函数 `cv2.connectedComponents()` 的标注结果进行修正。
7. 使用分水岭函数完成对图像的分割。

【例 17.6】 使用分水岭算法对一幅图像进行分割, 并观察分割的效果。

根据题目的要求, 编写代码如下:

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
img = cv2.imread('water_coins.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
imshow = img.copy()
ret, thresh = cv2.threshold(gray, 0, 255,
                             cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
kernel = np.ones((3, 3), np.uint8)
opening = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel, iterations = 2)
sure_bg = cv2.dilate(opening, kernel, iterations=3)
dist_transform = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
ret, sure_fg = cv2.threshold(dist_transform, 0.7 * dist_transform.max(), 255, 0)
sure_fg = np.uint8(sure_fg)
```



```

unknown = cv2.subtract(sure_bg, sure_fg)
ret, markers = cv2.connectedComponents(sure_fg)
markers = markers+1
markers[unknown==255] = 0
markers = cv2.watershed(img, markers)
img[markers == -1] = [0,255,0]
plt.subplot(121)
plt.imshow(ishow)
plt.axis('off')
plt.subplot(122)
plt.imshow(img)
plt.axis('off')

```

运行上述程序，得到的结果如图 17-13 所示。

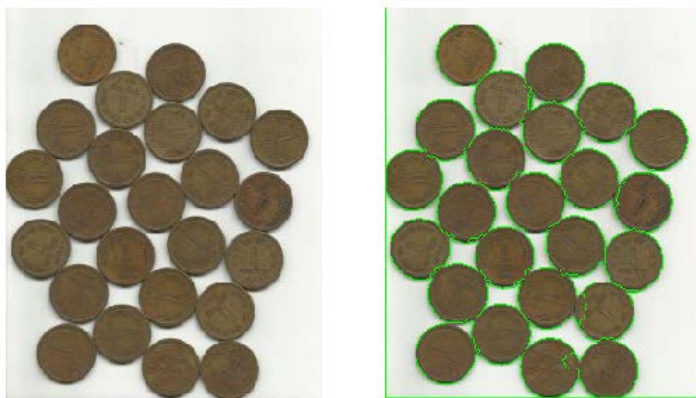


图 17-13 利用分水岭算法进行图像分割

17.2 交互式前景提取

经典的前景提取技术主要使用纹理（颜色）信息，如魔术棒工具，或根据边缘（对比度）信息，如智能剪刀等完成。2004 年，微软研究院（剑桥）的 Rother 等人在论文 *GrabCut: Interactive Foreground Extraction Using Iterated Graph Cuts* 中提出了交互式前景提取技术。他们提出的算法，仅需要做很少的交互操作，就能够准确地提取出前景图像。

在开始提取前景时，先用一个矩形框指定前景区域所在的大致位置范围，然后不断迭代地分割，直到达到最好的效果。经过上述处理后，提取前景的效果可能并不理想，存在前景没有提取出来，或者将背景提取为前景的情况，此时需要用户干预提取过程。用户在原始图像的副本中（也可以是与原始图像大小相等的任意一幅图像），用白色标注要提取为前景的区域，用黑色标注要作为背景的区域。然后，将标注后的图像作为掩模，让算法继续迭代提取前景从而得到最终结果。

例如，对于图 17-14 的左图，先用矩形框将要提取的前景 Lena 框出来，再分别用白色和黑色对前景图像、背景图像进行标注。完成标注后，使用交互式前景提取算法，就会得到图 17-14 右图所示的结果图像。

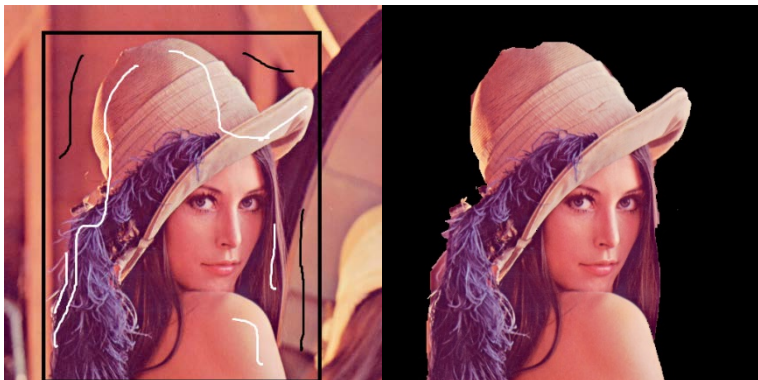


图 17-14 交互式前景提取示例

PowerPoint 2016 提供了“删除背景”功能。用户可以根据需要，在图像上标注出需要保留的部分和需要删除的部分，然后让 PowerPoint 帮助我们完成前景对象的提取。尝试在 PowerPoint 中删除图像背景，会帮助我们更好地理解交互式前景提取时模板的使用方式。

下面我们来看 GrabCut 算法的具体实施过程。

1. 将前景所在的大致位置使用矩形框标注出来。值得注意的是，此时矩形框框出的仅仅是前景的大致位置，其中既包含前景又包含背景，所以该区域实际上是未确定区域。但是，该区域以外的区域被认为是“确定背景”。
2. 根据矩形框外部的“确定背景”数据来区分矩形框区域内的前景和背景。
3. 用高斯混合模型（Gaussians Mixture Model, GMM）对前景和背景建模。GMM 会根据用户的输入学习并创建新的像素分布。对未分类的像素（可能是背景也可能是前景），根据其已与已知分类像素（前景和背景）的关系进行分类。
4. 根据像素分布情况生成一幅图，图中的节点就是各个像素点。除了像素点之外，还有两个节点：前景节点和背景节点。所有的前景像素都和前景节点相连，所有的背景像素都和背景节点相连。每个像素连接到前景节点或背景节点的边的权重由像素是前景或背景的概率来决定。
5. 图中的每个像素除了与前景节点或背景节点相连外，彼此之间还存在着连接。两个像素连接的边的权重值由它们的相似性决定，两个像素的颜色越接近，边的权重值越大。
6. 完成节点连接后，需要解决的问题变成了一幅连通的图。在该图上根据各自边的权重关系进行切割，将不同的点划分为前景节点和背景节点。
7. 不断重复上述过程，直至分类收敛为止。

OpenCV 的官网上有更详细的资料（<http://www.cs.ru.ac.za/research/g02m1682/>），读者有兴趣的话可以进一步学习。

在 OpenCV 中，实现交互式前景提取的函数是 `cv2.grabCut()`，其语法格式为：

```
mask, bgdModel, fgdModel = cv2.grabCut(img, mask, rect, bgdModel, fgdModel,
iterCount[, mode])
```

式中：



- `img` 为输入图像，要求是 8 位 3 通道的。
- `mask` 为掩模图像，要求是 8 位单通道的。该参数用于确定前景区域、背景区域和不确定区域，可以设置为 4 种形式。
 - `cv2.GC_BGD`：表示确定背景，也可以用数值 0 表示。
 - `cv2.GC_FGD`：表示确定前景，也可以用数值 1 表示。
 - `cv2.GC_PR_BGD`：表示可能的背景，也可以用数值 2 表示。
 - `cv2.GC_PR_FGD`：表示可能的前景，也可以用数值 3 表示。

在最后使用模板提取前景时，会将参数值 0 和 2 合并为背景（均当作 0 处理），将参数值 1 和 3 合并为前景（均当作 1 处理）。在通常情况下，我们可以使用白色笔刷和黑色笔刷在掩模图像上做标记，再通过转换将其中的白色像素设置为 0，黑色像素设置为 1。

- `rect` 指包含前景对象的区域，该区域外的部分被认为是“确定背景”。因此，在选取时务必确保让前景包含在 `rect` 指定的范围内；否则，`rect` 外的前景部分是不会被提取出来的。只有当参数 `mode` 的值被设置为矩形模式 `cv2.GC_INIT_WITH_RECT` 时，参数 `rect` 才有意义。其格式为 `(x, y, w, h)`，分别表示区域左上角像素的 `x` 轴和 `y` 轴坐标以及区域的宽度和高度。如果前景位于右下方，又不想判断原始图像的大小，对于 `w` 和 `h` 可以直接用一个很大的值。使用掩模模式时，将该值设置为 `none` 即可。
- `bgdModel` 为算法内部使用的数组，只需要创建大小为(1, 65)的 `numpy.float64` 数组。
- `fgdModel` 为算法内部使用的数组，只需要创建大小为(1, 65)的 `numpy.float64` 数组。
- `iterCount` 表示迭代的次数。
- `mode` 表示迭代模式。其可能的值与含义如表 17-3 所示。

表 17-3 `mode` 的值及含义

参数 <code>mode</code> 的值	含义
<code>cv2.GC_INIT_WITH_RECT</code>	使用矩形模板
<code>cv2.GC_INIT_WITH_MASK</code>	使用自定义模板。需要注意， <code>cv2.GC_INIT_WITH_RECT</code> 和 <code>cv2.GC_INIT_WITH_MASK</code> 能组合使用。所有 ROI 区域外（不在模板或矩形指定范围内）的像素会自动被处理为背景
<code>cv2.GC_EVAL</code>	修复模式
<code>cv2.GC_EVAL_FREEZE_MODEL</code>	使用固定模式

【例 17.7】 使用 GrabCut 算法提取图像的前景，并观察提取效果。

根据题目的要求，编写代码如下：

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
o = cv2.imread('lenacolor.png')
orgb=cv2.cvtColor(o,cv2.COLOR_BGR2RGB)
mask = np.zeros(o.shape[:2],np.uint8)
bgdModel = np.zeros((1,65),np.float64)
fgdModel = np.zeros((1,65),np.float64)
rect = (50,50,400,500)
cv2.grabCut(o,mask,rect,bgdModel,fgdModel,5,cv2.GC_INIT_WITH_RECT)
```

```

mask2 = np.where((mask==2)|(mask==0),0,1).astype('uint8')
ogc = o*mask2[:, :, np.newaxis]
ogc=cv2.cvtColor(ogc,cv2.COLOR_BGR2RGB)
plt.subplot(121)
plt.imshow(orgb)
plt.axis('off')
plt.subplot(122)
plt.imshow(ogc)
plt.axis('off')

```

运行上述程序，得到的结果如图 17-14 所示。

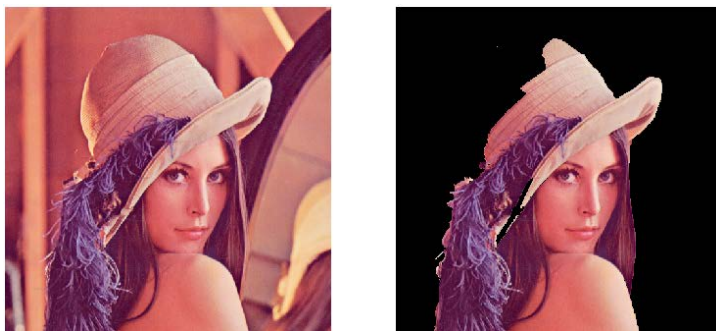


图 17-15 使用 GrabCut 算法提取前景

可以看到，在不使用掩模（掩模值都设置为默认值 0 时），函数 `cv2.grabCut()` 的处理效果并不太好：提取图 17-15 中左图的前景时，人物的帽子没有提取完整。对于有些图像，也有可能将背景错误地提取出来。

为了得到完整的前景对象，需要做一些改进。这里对原始图像进行标注，将需要保留的部分设置为白色，将需要删除的背景设置为黑色。以标记好的图像作为模板，使用函数 `cv2.grabCut()` 完成前景的提取。

这个过程主要包含以下步骤：

1. 利用函数 `cv2.grabCut()` 在 `cv2.GC_INIT_WITH_RECT` 模式下对图像进行初步的前景提取，得到初步提取的结果图像 `og`。
2. 使用 Windows 系统自带的笔刷工具，打开要提取前景的图像，比如 `lena`。
3. 使用白色笔刷在希望提取的前景区域做标记。
4. 使用黑色笔刷在希望删除的背景区域做标记。
5. 将当前设置好的 `lena` 图像另存为模板图像 `m0`。
6. 将模板图像 `m0` 中的白色值和黑色值映射到模板 `m` 中。将模板图像 `m0` 中的白色值（像素值为 255）映射为模板图像 `m` 中的确定前景（像素值为 1），将模板图像 `m0` 中的黑色值（像素值为 0）映射为模板图像 `m` 中的确定背景（像素值为 0）。
7. 以模板图像 `m` 作为函数 `cv2.grabCut()` 的模板参数（`mask`），对图像 `og` 完成前景提取。

需要注意，在上述步骤中，使用画笔标记的模板图像 `m0` 不能直接作为模板（即参数 `mask`）使用。函数 `cv2.grabCut()` 要求，参数 `mask` 的值必须是 `cv2.GC_BGD`（确定背景）、`cv2.GC_FGD`

(确定前景)、`cv2.GC_PR_BGD` (可能的背景)、`cv2.GC_PR_FGD` (可能的前景), 或者是 0、1、2、3 之中的值。此时的模板图像 `m0` 中, 存在着[0, 255]内的值, 所以它的值不满足函数 `cv2.grabCut()` 的要求, 无法作为参数 `mask` 直接使用。必须先将模板图像 `m0` 中的白色值和黑色值映射到模板 `m` 上, 再将模板图像 `m` 作为函数 `cv2.grabCut()` 的模板参数。

【例 17.8】 在 GrabCut 算法中使用模板提取图像的前景, 并观察提取的效果。

根据题目的要求, 编写代码如下:

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
o= cv2.imread('lenacolor.png')
orgb=cv2.cvtColor(o,cv2.COLOR_BGR2RGB)
mask = np.zeros(o.shape[:2],np.uint8)
bgd = np.zeros((1,65),np.float64)
fgd = np.zeros((1,65),np.float64)
rect = (50,50,400,500)
cv2.grabCut(o,mask,rect,bgd,fgd,5,cv2.GC_INIT_WITH_RECT)
mask2 = cv2.imread('mask.png',0)
mask2Show = cv2.imread('mask.png',-1)
m2rgb=cv2.cvtColor(mask2Show,cv2.COLOR_BGR2RGB)
mask[mask2 == 0] = 0
mask[mask2 == 255] = 1
mask, bgd, fgd = cv2.grabCut(o,mask,None,bgd,fgd,5,cv2.GC_INIT_WITH_MASK)
mask = np.where((mask==2)|(mask==0),0,1).astype('uint8')
ogc = o*mask[:, :, np.newaxis]
ogc=cv2.cvtColor(ogc,cv2.COLOR_BGR2RGB)
plt.subplot(121)
plt.imshow(m2rgb)
plt.axis('off')
plt.subplot(122)
plt.imshow(ogc)
plt.axis('off')
```

运行上述程序, 得到的结果如图 17-15 所示。

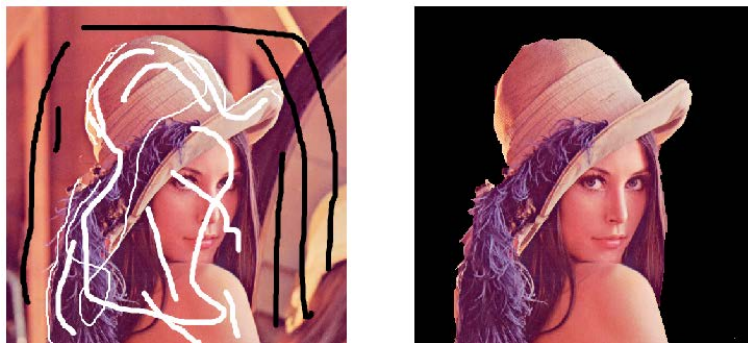


图 17-16 使用掩模提取前景

在函数 `cv2.grabCut()` 的实际使用中,也可以不使用矩形初始化,直接使用模板模式。构造一个模板图像,其中:

- 使用像素值 0 标注确定背景。
- 使用像素值 1 标注确定前景。
- 使用像素值 2 标注可能的背景。
- 使用像素值 3 标注可能的前景。

构造完模板后,直接将该模板用于函数 `cv2.grabCut()` 处理原始图像,即可完成前景的提取。

一般情况下,自定义模板的步骤为:

1. 先使用 `numpy.zeros` 构造一个内部像素值都是 0 (表示确定背景) 的图像 `mask`, 以便在后续步骤中逐步对该模板图像进行细化。
2. 使用 `mask[30:512, 50:400]=3`, 将模板图像中第 30 行到第 512 行, 第 50 列到 400 列的区域划分为可能的前景 (像素值为 3, 对应参数 `mask` 的含义为“可能的前景”)。
3. 使用 `mask[50:300, 150:200]=1`, 将模板图像中第 50 行到第 300 行, 第 150 列到第 200 列的区域划分为确定前景 (像素值为 1, 对应参数 `mask` 的含义为“确定前景”)。

在此基础上,将 `mask` 作为自定义模板,使用 `GrabCut` 算法,完成前景的提取。

【例 17.9】 在 `GrabCut` 算法中直接使用自定义模板提取图像的前景,并观察提取效果。

根据题目的要求,编写代码如下:

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
o= cv2.imread('lena_color.png')
orgb=cv2.cvtColor(o,cv2.COLOR_BGR2RGB)
bgd = np.zeros((1,65),np.float64)
fgd = np.zeros((1,65),np.float64)
mask2 = np.zeros(o.shape[:2],np.uint8)
#先将掩模的值全部构造为 0 (确定背景),在后续步骤中,再根据需要修改其中的部分值
mask2[30:512,50:400]=3      #lena 头像的可能区域
mask2[50:300,150:200]=1     #lena 头像的确定区域,如果不设置这个区域,头像的提取不完整
cv2.grabCut(o,mask2,None,bgd,fgd,5,cv2.GC_INIT_WITH_MASK)
mask2 = np.where((mask2==2)|(mask2==0),0,1).astype('uint8')
ogc = o*mask2[:, :, np.newaxis]
ogc=cv2.cvtColor(ogc,cv2.COLOR_BGR2RGB)
plt.subplot(121)
plt.imshow(orgb)
plt.axis('off')
plt.subplot(122)
plt.imshow(ogc)
plt.axis('off')
```

运行上述程序,得到的结果如图 17-16 所示。



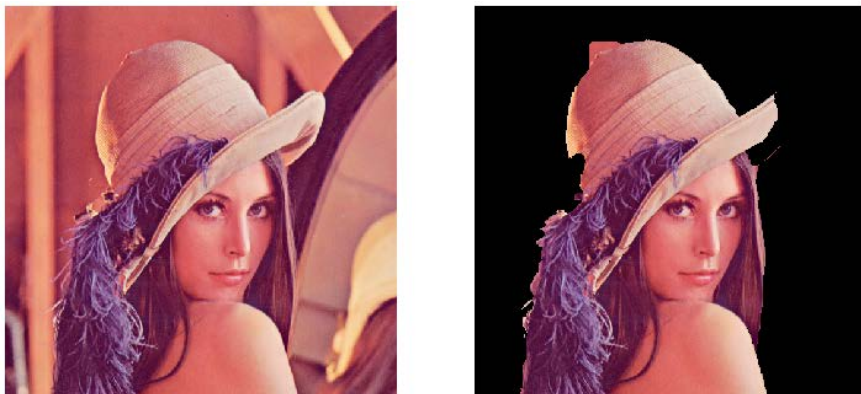


图 17-17 使用自定义模板提取前景

为了方便大家理解，本例中构造的模板比较简单。如果构造更有针对性的模板，能够提取更为准确的前景对象。

需要注意的是，对于不同的图像，要构造不同的模板来划分它们的确定前景、确定背景、可能的前景与可能的背景。